

NUMB11: Numerical Linear Algebra: Assignment 3

Arpita Bhattacharya
20010823-7348

September 23, 2024

1. Task 1 solution:

In this task, a model of a signal is given by the formula:

$$y(x) = a \sin(x) + b \cos(x) + c \sin(2x) + d \cos(2x) \quad (1)$$

with unknown coefficients a, b, c, d . To determine these coefficients we are asked to take a least squares approach.

Let's assume the measured signal is stored in an array of values

$$\mathbf{y} = [y_1, y_2, \dots, y_{200}] \quad (2)$$

where each y_i is the signal at each of the 200 equidistant sampling points in $[0, 4\pi]$. These are given in data file `signal.dat`.

This model can be written as the system of linear equations:

$$\mathbf{y} = a\mathbf{f}_1 + b\mathbf{f}_2 + c\mathbf{f}_3 + d\mathbf{f}_4 \quad (3)$$

where,

$$\begin{aligned} \mathbf{f}_1 &= [\sin(x_1), \sin(x_2), \dots, \sin(x_{200})]^T \\ \mathbf{f}_2 &= [\cos(x_1), \cos(x_2), \dots, \cos(x_{200})]^T \\ \mathbf{f}_3 &= [\sin(2x_1), \sin(2x_2), \dots, \sin(2x_{200})]^T \\ \mathbf{f}_4 &= [\cos(2x_1), \cos(2x_2), \dots, \cos(2x_{200})]^T \end{aligned} \quad (4)$$

Thus, the signal can be expressed as a matrix form:

$$\mathbf{y} = \mathbf{A}\mathbf{F} \quad (5)$$

where $\mathbf{y}$ is the vector of all the measured signals, $\mathbf{F}$ is the vector of unknown coefficients a, b, c, d and $\mathbf{A}$ is a 200×4 matrix consisting of the RHS of equation 4.

To find the unknown coefficients a, b, c, d , we need to solve the least squares problem by minimizing the residual:

$$\arg \min_{\mathbf{F}} \|\mathbf{y} - \mathbf{A}\mathbf{F}\|_2 \quad (6)$$

The solution of this can be found through a variety of methods such as normal equations, QR-factorization, SVD (Singular Value Decomposition), etc. Here, we will solve it separately first by QR-factorization, then by SVD and compare the results along with the computational time.

The QR- factorization code and SVD code written in python is given in the appendix 1.1.1 along with the output I received (Appendix: 1.1.2).

We can see that both methods gave us the same results. But comparing the computational time, we can see that the result is computed faster using the SVD method compared to QR-factorization.

Finally, I have plotted signal. There are 3 plots in one (one plotted using the measured data, one using the results from QR-factorization and one using the results from the SVD method) (Appendix: 1.1.2). But, only one plot is visible as they all yield the same results. So the plots are superimposed over one another.

2. Task 2 solution:

The condition number of a matrix can give a sharp estimate of the sensitivity of x with respect to perturbations of b when solving $Ax = b$, this means there exists a right hand side b and a perturbation δb such that

$$\frac{\|\delta x\|_2}{\|x\|_2} = \kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2} \quad (7)$$

In this task, we need to find a vector pair $(b, \delta b)$ for which this equality holds.

To find this, we need to consider the SVD (Singular Value Decomposition) of the matrix A .

We know the matrix A can be decomposed as $A = U\Sigma V^T$ where, U is an orthogonal matrix with left singular vectors $u_1, u_2, \dots, u_n$, V is an orthogonal matrix with right singular vectors $v_1, v_2, \dots, v_n$, and Σ is a diagonal matrix of the singular values $\sigma_1, \sigma_2, \dots, \sigma_n$ such that, $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$.

The condition number of a matrix A with respect to 2-norm is given by:

$$\kappa(A) = \frac{\sigma_1}{\sigma_n} \quad (8)$$

where σ_1 is the largest singular value and σ_n is the smallest singular value.

Next, to find vectors b and δb , we can express them in terms of the left singular vectors of A .

Let $b = u_1$, where u_1 is the left singular vector corresponding to the largest singular value σ_1 and let $\delta b = u_n$, where u_n is the left singular vector corresponding to the smallest singular value σ_n .

This is because the perturbation δb is chosen such that it would be in the direction in which A is least sensitive while b is chosen such that it would be in the direction in which A is most sensitive.

We can verify the choice:

For b , $Ax = b = u_1$.

Since $Av_1 = \sigma_1 u_1$, then $x = \frac{1}{\sigma_1} v_1$.

So, $\|x\|_2 = \frac{1}{\sigma_1}$

For δb , $A\delta x = \delta b = u_n$.

Since $Av_n = \sigma_n u_n$, then $\delta x = \frac{1}{\sigma_n} v_n$.

So, $\|\delta x\|_2 = \frac{1}{\sigma_n}$

Thus,

In the equation $\frac{\|\delta x\|_2}{\|x\|_2} = \kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2}$,

LHS:

$$\frac{\|\delta x\|_2}{\|x\|_2} = \frac{\frac{1}{\sigma_n}}{\frac{1}{\sigma_1}} = \frac{\sigma_1}{\sigma_n} = \kappa(A) \quad (9)$$

RHS:

$$\kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2} = \kappa_2(A) \frac{\|u_n\|_2}{\|u_1\|_2} = \kappa_2(A)(1) = \kappa_2(A) \quad (10)$$

Thus, LHS = RHS.

Hence, we have verified that the choice $b = u_1$ and $\delta b = u_n$ where u_1 and u_n are the left singular vectors corresponding to the largest and smallest singular values respectively, satisfies the equality.

3. Task 3 solution:

This task is regarding Hilbert matrices.

In the code 1.2.1, we define a 50×50 Hilbert matrix H along with the condition number of the matrix H .

We know that $\kappa(H) = \frac{\sigma_1}{\sigma_n}$ where σ_1 and σ_n are the largest and smallest singular values of H .

The condition number ($\kappa_2(H)$) tells us how sensitive the solution x is to changes in b when solving $Ax = b$. For ill-conditioned matrices like the Hilbert matrix, this number is very large. Here, we find the 2-norm condition number of the Hilbert matrix.

Worst-case b and δb :

- $b = u_1$ (first left singular vector): This corresponds to $Hv_1 = \sigma_1 u_1$

- $\delta b = u_n$ (last left singular vector): This corresponds to $Hv_n = \sigma_n u_n$

Using the fact that H is symmetric and positive definite, the right singular vectors (v) are the same as the left singular vectors (u). v_1 and v_n are approximately aligned with the first and last column

of the inverse of H .

We set b to the first column of the inverse Hilbert matrix, which approximates the vector corresponding to the largest singular value (associated with u_1 , the left singular vector corresponding to σ_1). And, we set δb to the last column of the inverse Hilbert matrix, which approximates the vector corresponding to the smallest singular value (associated with u_n , the left singular vector corresponding to σ_n).

We then solve two linear systems:

$$\begin{aligned} Hx &= b \\ H\delta x &= \delta b \end{aligned} \tag{11}$$

We finally find $\frac{\|\delta x\|_2}{\|x\|_2}$ and $\kappa_2(H) \frac{\|\delta b\|_2}{\|b\|_2}$. The result can be viewed in the appendix 1.2.2.

From Task 2, we know that there exists vectors b and δb such that:

$$\frac{\|\delta x\|_2}{\|x\|_2} = \kappa_2(H) \frac{\|\delta b\|_2}{\|b\|_2} \tag{12}$$

This means that in the worst-case scenario, the relative change in x due to a perturbation in b is amplified by the condition number $\kappa_2(H)$. The goal is to check whether this relationship holds using the Hilbert matrix H .

We have got:

$$\begin{aligned} \frac{\|\delta x\|_2}{\|x\|_2} &= 2.86 \times 10^{28} \\ \kappa_2(H) &= 2.31 \times 10^{19} \quad \text{and hence,} \\ \kappa_2(H) \times \frac{\|\delta b\|_2}{\|b\|_2} &= 4.90 \times 10^{47} \end{aligned} \tag{13}$$

Comparing the values of $\frac{\|\delta x\|_2}{\|x\|_2}$ and $\kappa_2(H) \frac{\|\delta b\|_2}{\|b\|_2}$, we can observe that these two quantities are vastly different, with the right-hand side of 12 being much larger than the left-hand side.

Thus, the result from the code does not verify the strict equality from Task 2.

4. Task 4 solution:

In this task we need to show that a strictly diagonally dominant matrix A is invertible. The idea is to show that there is no $u \neq 0$ which satisfies $Au = 0$.

Let's first assume that $Au = 0$ for some $u = [u_1, u_2, \dots, u_n]^T \neq 0$ where $A \in \mathbb{R}^{n \times n}$ is a strictly diagonally dominant matrix.

This implies that,

$$\sum_{j=1}^n a_{ij}u_j = 0 \quad (14)$$

or,

$$a_{ii}u_i + \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}u_j = 0 \quad (15)$$

Since we assumed that $u \neq 0$, then at least one element of u is non-zero.

If all elements u_i of u had the same absolute value, $|u_1| = |u_2| = \dots = |u_n|$, then for each row i ,

$$\begin{aligned} a_{ii}u_i + \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}u_j &= 0 \\ |a_{ii}||u_i| &= \left| \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}u_j \right| && \text{Taking absolute value} \\ |a_{ii}||u_i| &\leq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}||u_j| && \text{Triangle inequality} \\ |a_{ii}||u_i| &\leq |u_j| \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| && |u_i| = |u_j| \text{ for all } j \\ |a_{ii}| &\leq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \end{aligned} \quad (16)$$

But, this contradicts the strict diagonal dominance condition where $|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$.

Thus, it is not possible for all elements of u to have the same absolute value.

Next, if there is an index k such that $|u_k| \geq |u_j|$ for all j . WLOG we'll assume $|u_k| > 0$ as $u \neq 0$.

Now similarly for each row k ,

$$\begin{aligned}
a_{kk}u_k + \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj}u_j &= 0 \\
|a_{kk}||u_k| &= \left| \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj}u_j \right| && \text{Taking absolute value} \\
|a_{kk}||u_k| &\leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}||u_j| && \text{Triangle inequality} \\
|a_{kk}||u_k| &\leq |u_k| \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| && |u_k| \leq |u_j| \text{ for all } j \neq k \\
|a_{kk}| &\leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|
\end{aligned} \tag{17}$$

But, this also contradicts the strict diagonal dominance condition where $|a_{kk}| > \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|$.

Thus, it is not possible for any element u_i of u to satisfy $|u_k| \geq |u_j|$ for all j .

Hence, we have shown that there can't be any u_i in u such that $|u_k| \geq |u_j|$ for all j and neither can all u_i be the same absolute value.

This implies that there is no such $u \neq 0$ which satisfies $Au = 0$. Or, the only solution this has is $u = 0$. This is a contradiction.

This would mean that A has no non-trivial null space and thus, A is invertible.

5. Task 5 solution:

This task is about discretizing the given differential equation $-u''(x) = f(x)$ over $[0, 1]$ with boundary conditions $u(0) = u(1) = 0$.

To discretize the equation, we approximate the second derivative using finite differences. This is done using the central difference formula. For $n + 1$ equidistant points $x_i = \frac{i}{n}$, the second derivative at x_i can be approximated as:

$$-u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} \tag{18}$$

where $\Delta x = \frac{1}{n}$. Hence,

$$-\frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} = f(x_i) \tag{19}$$

Or,

$$u_{i-1} - 2u_i + u_{i+1} = -\Delta x^2 f(x_i) \tag{20}$$

Next, we see that the equation 20 forms a linear system for $i = 1, 2, \dots, n-1$. The boundary conditions $u(0) = u(1) = 0$ imply that $u_0 = u_n = 0$. Hence, the linear system can be written as:

$$\begin{aligned} -2u_1 + u_2 &= -\Delta x^2 f(x_1) & i = 1 \\ u_{i-1} - 2u_i + u_{i+1} &= -\Delta x^2 f(x_i) & i = 2, \dots, n-2 \\ u_{n-2} - 2u_{n-1} &= -\Delta x^2 f(x_{n-1}) \end{aligned} \quad (21)$$

This system can be re-written as:

$$\begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & -2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} -\Delta x^2 f(x_1) \\ -\Delta x^2 f(x_2) \\ -\Delta x^2 f(x_3) \\ \vdots \\ -\Delta x^2 f(x_{n-1}) \end{bmatrix} \quad (22)$$

Hence the matrix representing this system is:

$$\begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & -2 \end{bmatrix}.$$

This is a tri-diagonal matrix. Not only is it tri-diagonal, but it is diagonally dominant. The determinant of a diagonally dominant tri-diagonal matrix with non-zero diagonal elements is non-zero. Hence, this matrix is **non-singular**.

Finally the matrix that is given to us in the task,

$$\begin{bmatrix} 1 & -1 & 0 & & \\ -1 & 2 & -1 & \ddots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ & \ddots & -1 & 2 & 1 \\ & & 0 & -1 & 1 \end{bmatrix} \quad (23)$$

This is also a diagonally dominant tri-diagonal matrix with non-zero diagonal elements. Hence, its determinant is also non-zero.

Thus, this matrix is also **non-singular**.

1 Appendix

1.1 Task 1

1.1.1 Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
import time

# Load the data
data = np.loadtxt('signal.dat', delimiter=',')
x_values = data[:, 0] # First column: x values
y_values = data[:, 1] # Second column: y values (measured signal)

# Construct the design matrix A
A = np.column_stack([
    np.sin(x_values),      # First column: sin(x)
    np.cos(x_values),      # Second column: cos(x)
    np.sin(2 * x_values),  # Third column: sin(2x)
    np.cos(2 * x_values)   # Fourth column: cos(2x)
])

### QR Factorization Approach
start_qr = time.time()

# Step 1: Compute reduced QR factorization
Q, R = np.linalg.qr(A)

# Step 2: Compute  $Q^T * b$ 
b_qr = Q.T @ y_values

# Step 3: Solve the upper triangular system  $R * x = Q^T * b$ 
x_qr = np.linalg.solve(R, b_qr)

end_qr = time.time()
time_qr = end_qr - start_qr

### SVD Approach
start_svd = time.time()

# Step 1: Compute reduced SVD,  $A = US(V^T)$ 
U, S, Vt = np.linalg.svd(A, full_matrices=False)

# Step 2: Compute  $U^T * b$ 
b_svd = U.T @ y_values

# Step 3: Solve the diagonal system  $S * w = U^T * b$ 
w_svd = b_svd / S # S is diagonal, so we can divide element-wise

# Step 4: Compute  $x = V * w$ 
x_svd = Vt.T @ w_svd

end_svd = time.time()
time_svd = end_svd - start_svd

### Comparing the results and computational time
print(f"Coefficients (QR Factorization): {x_qr}")
```



```

print(f"Coefficients (SVD): {x_svd}")
print(f"Time taken (QR Factorization): {time_qr:.6f} seconds")
print(f"Time taken (SVD): {time_svd:.6f} seconds")

# Reconstruct the signal using both methods
y_qr = A @ x_qr
y_svd = A @ x_svd

### Plotting the signal
plt.figure(figsize=(10, 6))

# Original signal (measured data)
plt.plot(x_values, y_values, label='Measured Signal', color='blue', linestyle='--')

# Reconstructed signal using QR
plt.plot(x_values, y_qr, label='Reconstructed Signal (QR)', color='green')

# Reconstructed signal using SVD
plt.plot(x_values, y_svd, label='Reconstructed Signal (SVD)', color='red')

plt.title('Measured vs Reconstructed Signal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

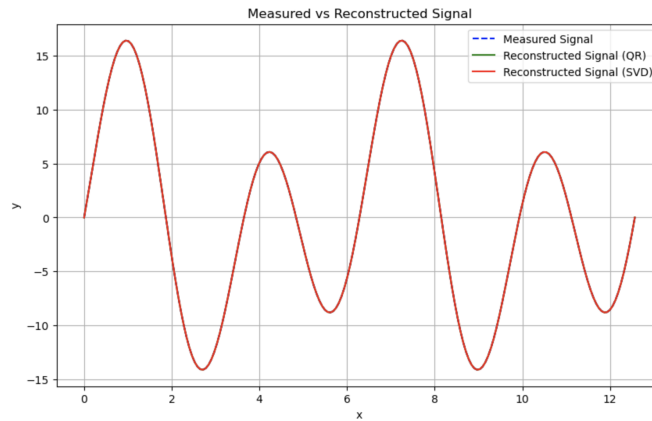
```

1.1.2 Output:

```

Coefficients (QR Factorization): [ 3.00005803  4.9998545  9.99992389 -4.99997327]
Coefficients (SVD): [ 3.00005803  4.9998545  9.99992389 -4.99997327]
Time taken (QR Factorization): 0.002957 seconds
Time taken (SVD): 0.000899 seconds

```



1.2 Task 3

1.2.1 Python Code:

```
import numpy as np
from scipy.linalg import hilbert, invhilbert

# Define the size of the Hilbert matrix
n = 50

# Construct the Hilbert matrix and its inverse
H = hilbert(n)
H_inv = invhilbert(n)

# Compute the condition number of the Hilbert matrix in the 2-norm
cond_H = np.linalg.cond(H, 2)

# Define worst-case b and delta_b
b = H_inv[:, 0]
delta_b = H_inv[:, -1]

# Solve the systems Hx = b and H(delta_x) = delta_b
x = np.linalg.solve(H, b)
delta_x = np.linalg.solve(H, delta_b)

# Compute the relative perturbations in x and b
relative_x_perturbation = np.linalg.norm(delta_x) / np.linalg.norm(x)
relative_b_perturbation = np.linalg.norm(delta_b) / np.linalg.norm(b)

# Expected worst-case amplification
worst_case_amplification = cond_H * relative_b_perturbation

relative_x_perturbation, worst_case_amplification, cond_H
```

1.2.2 Output:

```
(2.8606582500833184e+28, 4.903340129493973e+47, 2.3125456566766473e+19)
```